

Feature/RAG Branch Summary

This document summarizes the practical RAG work implemented in the feature/rag branch. It is intended as a quick understanding guide, not a full technical specification.

Current Status

Area	Status
Backend RAG flow	Working with report-scoped retrieval and structured responses
Frontend	Build and lint pass; chat still uses non-streaming endpoint
Streaming endpoint	Implemented; fallback path validated
Real embeddings	Supported in code, not fully validated end-to-end

What Was Implemented

- Report-scoped RAG retrieval using report_id metadata and report-specific Chroma collections.
- Hybrid retrieval combining semantic search with BM25 keyword reranking when available.
- Query routing for factual, analytical, and comparative questions.
- Reranking and MMR-style filtering to improve relevance and reduce duplicate context.
- Conversation memory using report_id + session_id, including simple follow-up query contextualization.
- Structured financial prompt templates that enforce citations and insufficient-evidence behavior.
- Citation verification, hallucination checks, confidence scores, and structured RAG response metadata.
- RAG diagnostics endpoint for collection stats, embedding mode, reranker status, and sample retrieval.
- Streaming chat endpoint that uses Ollama streaming when available and falls back safely when unavailable.
- Section-scoped parent-child chunking with char ranges, parent chunk IDs, section names, and duplicate sentence counts.
- Frontend fixes for TypeScript build, lint cleanup, and chat session_id persistence.
- Backend dependency manifest through requirements.txt.

Main Files Added or Changed

- backend/rag/hybrid_search.py, reranker.py, query_router.py
- backend/rag/conversation_memory.py, prompt_templates.py
- backend/rag/citation_verifier.py, hallucination_guard.py, answer_formatter.py
- backend/rag/rag_diagnostics.py and retrieval.py
- backend/tools/chroma_tool.py and embedding_tool.py
- backend/graph/langgraph_flow.py and backend/main.py
- backend/rag/chunking.py
- frontend/app/page.tsx
- requirements.txt

Validation Completed

- Backend compile check passed: `python -m compileall backend`.
- Frontend lint passed: `npm run lint`.
- Frontend production build passed: `npm run build`.
- Smoke tested upload pipeline with synthetic filing text.
- Smoke tested `/api/chat` with cited, structured RAG response.
- Smoke tested `/api/chat/stream` and verified final metadata packet.
- Smoke tested `/api/rag/diagnostics/{report_id}` and verified chunk stats.

Remaining Gaps

- Real BAAI embeddings are supported but were not validated end-to-end because the model is large.
- Ollama live token streaming code exists, but local Ollama timed out during validation; fallback streaming was validated.
- Frontend does not yet consume `/api/chat/stream`; it still uses `/api/chat`.
- Citation and hallucination checks are heuristic, not full NLI/entailment systems.
- No formal automated pytest or frontend test suite was added yet.
- Chunk metadata does not yet include reliable PDF page numbers or filing dates.

Commits Created

- 14931cf - fix: stabilize scoped rag pipeline
- 19abf76 - feat: complete rag streaming and chunk scoping

Bottom line: the branch now has a working backend RAG implementation with scoped retrieval, citations, memory, diagnostics, and streaming support. It is usable, but the final production-grade polish requires real model validation, frontend streaming integration, and automated tests.